

Comparing Memory Overhead and Lookup Runtime of Common Python Data Structures

Bachelor of Science in Computer Science

Programming Languages Case Study

Mark Renier B. Fostanes

mfostanes@ssct.edu.ph

Experiment settings: $n = 10,000$ | repeat = 10,000 | target = $n - 1$

Abstract

This paper compares Python list, tuple, set, and dictionary in terms of memory overhead and lookup runtime. Using `sys.getsizeof()` and `time.perf_counter()`, the experiment shows that tuple is slightly more memory-efficient than list, while set and dictionary use more memory but achieve much faster membership testing. The study highlights the trade-off between storage cost and execution speed when selecting a Python data structure.

1. Introduction

Python provides several built-in data structures for storing and accessing values. Although these structures may hold similar data, they differ in both memory consumption and execution behavior. In small programs the difference may appear minimal, but in larger applications the choice of data structure can affect resource usage and overall performance. This study focuses on two measurable factors: memory overhead and lookup runtime.

2. Objective of the Study

This case study compares the memory overhead and lookup runtime of list, tuple, set, and dictionary. It aims to identify which structure is more memory-efficient and which offers faster membership testing under the same experimental conditions.

3. Methodology

The program creates four data structures using the same numeric range from 0 to 9,999. Memory usage is measured using `sys.getsizeof()`, while lookup runtime is measured using `time.perf_counter()`. The membership test target is the last value in the collection, and each lookup is repeated 10,000 times to make the timing results clearer and more stable.

4. Experimental Code

Figure 1 presents the exact Python implementation used in the experiment.

```
1  import sys
2  import time
3
4  n = 10000
5  target = n - 1
6  repeat = 10000
7
8  my_list = list(range(n))
9  my_tuple = tuple(range(n))
10 my_set = set(range(n))
11 my_dict = {i: i for i in range(n)}
12
13 print("Memory Usage:")
14 print("List:", sys.getsizeof(my_list), "bytes")
15 print("Tuple:", sys.getsizeof(my_tuple), "bytes")
16 print("Set:", sys.getsizeof(my_set), "bytes")
17 print("Dictionary:", sys.getsizeof(my_dict), "bytes")
18
19 start = time.perf_counter()
20 for _ in range(repeat):
21     target in my_list
22 end = time.perf_counter()
23 print(f"\nList search time: {(end - start):.8f} seconds")
24
25 start = time.perf_counter()
26 for _ in range(repeat):
27     target in my_tuple
28 end = time.perf_counter()
29 print(f"Tuple search time: {(end - start):.8f} seconds")
30
31 start = time.perf_counter()
32 for _ in range(repeat):
33     target in my_set
34 end = time.perf_counter()
35 print(f"Set search time: {(end - start):.8f} seconds")
36
37 start = time.perf_counter()
38 for _ in range(repeat):
39     target in my_dict
40 end = time.perf_counter()
41 print(f"Dictionary search time: {(end - start):.8f} second
    s")
```

Figure 1. Python code used to compare memory usage and search runtime across the four data structures.

5. Execution Output and Measured Results

Figure 2 shows the actual program output captured after execution in PowerShell.

```
PS C:\Users\fosta\OneDrive\Desktop\Programming Languages\Case Study> python case_study.py
Memory Usage:
List: 80056 bytes
Tuple: 80040 bytes
Set: 524504 bytes
Dictionary: 295000 bytes

List search time: 0.89015680 seconds
Tuple search time: 0.74386040 seconds
Set search time: 0.00069220 seconds
Dictionary search time: 0.00115540 seconds
PS C:\Users\fosta\OneDrive\Desktop\Programming Languages\Case Study> |
```

Figure 2. Actual memory and runtime measurements produced by the Python program.

Table 1. Summary of observed results

Structure	Memory (bytes)	Search time (s)	Interpretation
List	80,056	0.89015680	Low overhead, but slow linear search
Tuple	80,040	0.74386040	Smallest memory among the four; still linear lookup
Set	524,504	0.00069220	Highest memory cost, but very fast hashed lookup
Dictionary	294,992	0.00115540	Higher memory usage with fast key membership testing

6. Discussion

- Tuple used slightly less memory than list because it is immutable and carries less structural overhead for modification.
- List and tuple membership testing took much longer because both perform sequential search when checking whether the target value exists.
- Set and dictionary were dramatically faster because they use hashing, which allows near-constant-time lookup in average cases.
- The results show a clear trade-off: structures optimized for fast lookup generally require more memory.

7. Conclusion

The experiment shows that common Python data structures behave differently even when they store the same logical dataset. Tuple was the most memory-efficient structure among those tested, while set and dictionary consumed more memory but provided much faster lookup performance. Therefore, the choice of data structure should depend on the goal of the program: conserving memory, enabling updates, or improving search speed.

8. Limitation

This study reports one environment-specific run. Exact byte counts and timings may vary across Python versions, operating systems, and hardware. In addition, `sys.getsizeof()` measures container size directly and does not fully represent all nested memory costs. Still, the comparison remains useful for basic analysis of relative behavior.